

Basic (your first real API call) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real HTTP client tells failure types apart). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You're doing two separate things here: make one API call that works, then make one API call that's broken on purpose.

The `requests` library gives you a `.get(url)` function. Call it on a real, working address, and look at what comes back — it has a `.status_code` (a number like 200) and a `.json()` method that turns the reply into a Python dictionary.

Then break the URL on purpose — a typo in the protocol, like `htp://` instead of `http://` — and see what Python does differently. It won't give you back a response at all; it'll raise an error before it even reaches the network.

Things to use:

- `requests.get("https://api.github.com")` — makes the call.
- `.status_code` — the number, like `200`.
- `.json()` — turns the body into a Python dict.
- `try: ... except Exception as e: print(type(e).__name__)` — catches whatever goes wrong and prints its name.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about seeing two different failure categories with your own eyes, not just reading about them.

A **successful call** gives you a `requests.Response` object — `.status_code: int` and `.json() -> dict | list` (it parses the body as JSON for you, raising `json.JSONDecodeError` if the body isn't valid JSON).

A **malformed URL** never reaches the network at all — `requests` raises `requests.exceptions.MissingSchema` (or a similar `requests.exceptions.RequestException` subclass) immediately, client-side, before any bytes go anywhere. This is a completely different kind of failure than a bad *response* — nothing was sent, so there's nothing to receive.

The exact pieces:

- `requests.get(url) -> requests.Response` — the return type is always a `Response` object on success, never `None`.
- `requests.exceptions.RequestException` — the base class for everything `requests` can raise client-side (bad URL, connection refused, timeout, etc.). Catching this specific base — not bare `Exception` — is the right habit once you get past this exercise.
- `type(e).__name__` — every exception instance knows its own class; this is how you inspect exactly what you caught without guessing.

Write both blocks — the working call and the broken one — before checking Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A malformed URL is only one of several ways a call can fail before you ever get a `Response` back — a real client also has to tell apart a DNS lookup that fails (`requests.exceptions.ConnectionError`), a server that refuses the connection outright (also `ConnectionError`), and a certificate that doesn't validate (`requests.exceptions.SSLError`). Catching only `MissingSchema`, or catching bare `Exception`, means you learn nothing about *which* of these happened — and each one points you toward a different fix.

There's a second, easy-to-miss failure mode too: `requests.get(url)` can succeed and still hand you a `Response` carrying a 4xx or 5xx status — that's not an exception at all, just a "successful" call that failed at the HTTP level. `response.raise_for_status()` is what turns that into a catchable exception (`requests.exceptions.HTTPError`) too, so your `except` block can handle "the request itself failed" and "the server said no" the same way, instead of writing a separate `if response.status_code >= 400:` check every time.

The real design question isn't "how do I catch the error" — it's "how many different exception types am I willing to tell apart, and what does my code do differently for each one?" A script exploring an API by hand can get away with one broad `except RequestException`. A client meant to run unattended needs to know the difference between "this URL will never work, don't retry" and "the server is just having a bad moment."

The extra piece worth trying:

- `response.raise_for_status()` — call it right after `requests.get(...)` succeeds; it raises `requests.exceptions.HTTPError` if the status is 4xx or 5xx, and does nothing if the call actually succeeded. Wrap both the `.get()` call and this line in the same `try/except requests.exceptions.RequestException` block, since `HTTPError` is itself a subclass of it.

Difference between Basic, Intermediate, and Advanced: Basic names the tools for the two obvious outcomes (a working call, a broken URL). Intermediate names the real exception class involved and the habit of catching it specifically. Advanced points out that "broken URL" is just one of several distinct client-side failures (DNS, connection refused, TLS), and that a "successful" call can still carry a failing status code — which `raise_for_status()` folds into the same exception-handling path instead of a separate `if` check, closer to what an actual HTTP client library does under the hood.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
call requests.get on a real URL
print the status code
print the parsed JSON

try:
    call requests.get on a broken URL
except any error as e:
    print the error's class name
```

Here's almost the whole thing:

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
```

Expected output if you run just this: a number like `200` , printed once. What's missing: printing `response.json()` , and the whole second block with the broken URL and `try/except` . Add those yourself, then check Hint's Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except requests.exceptions.RequestException as e:
    print(type(e).__name__, "-", e)
```

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except requests.exceptions.RequestException as e:
    ... # what goes here?
```

Fill in the `except` block yourself, then compare against the [Solution](#). Notice the working call needs no `try/except` at all for this exercise — you're only wrapping the call you already know will fail, to see exactly how it fails.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
try:
    response = requests.get(a real URL)
    response.raise_for_status()
except requests.exceptions.RequestException as e:
    print(type(e).__name__, "-", e)
else:
    print(response.status_code)
    print(response.json())

for each broken url in [a bad schema, a URL with no such host, a URL that 404s]:
    try:
        response = requests.get(broken url)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(broken url, "->", type(e).__name__)
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# api_first_call_practice.py
import requests

urls = [
    "https://api.github.com/this-path-does-not-exist-at-all",
    "https://this-domain-genuinely-does-not-exist.invalid",
    "http://broken",
]

for url in urls:
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        # your turn: print the url and type(e).__name__ so you can see
        # that all 3 of these very different failures still land in
        # this one except block, but with 3 different exception classes
        ...
```

Fill in the `except` block, run it, and look at the 3 different exception class names it prints, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same idea (try a call, watch it fail) at 3 completeness levels — Basic proves a call works and a broken one raises something. Intermediate names the specific exception class and catches only that. Advanced adds `raise_for_status()` so a "successful" 404 is caught the same way as a connection failure, and runs several different kinds of broken URLs through the same block to show that `RequestException` is really an umbrella over several distinct, more specific failures.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)